HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO MÔN HỌC
PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Agent 02

Họ tên: Phan Văn Khôi
Mã sinh viên: B23DCCN447
Lớp: D23CTPM01-B
Giảng viên PGS. TS. Trần Đình Quế

Hà Nội – 2026

Mục lục

1 Giới thiệu và mục tiêu **1**

2 Môi trường và hai agent **2**

2.1 Trạng thái, hành động và động lực môi trường 2

2.2 Agent A: Q-learning dạng bảng 3

2.3 Agent B: chính sách dựa trên luật 3

2.4 Phân tích lợi nhuận kỳ vọng một bước 3

3 Cài đặt và thiết lập thí nghiệm **5**

3.1 Kiến trúc chương trình 5

3.2 Các thay đổi so với mã tham khảo 5

3.3 Tham số và quy trình đánh giá 6

3.4 Trích đoạn mã nguồn chính 7

3.4.1 Mô hình thị trường và môi trường 7

3.4.2 Agent Q-learning 8

3.4.3 Agent dựa trên luật 9

3.4.4 Vòng huấn luyện 9

3.4.5 Đánh giá tách biệt và đánh giá chính xác 10

4 Kết quả thí nghiệm **13**

4.1 Thí nghiệm A: baseline 13

4.2 Thí nghiệm B: quá trình học 14

4.3 Thí nghiệm C: tốc độ học 16

4.4 Thí nghiệm D: mức khám phá 18

4.5 Thí nghiệm E: so sánh công bằng trên nhiều seed 19

5 Trả lời mười câu hỏi **21**

Câu 1. Mô hình hóa agent và môi trường 21

Câu 2. Quyết định dựa trên học và dựa trên luật 22
Câu 3. Cập nhật Q-learning 23
Câu 4. Khám phá và khai thác 23
Câu 5. Phần thưởng và mục tiêu kinh doanh 24
Câu 6. Đánh giá hiệu năng 24
Câu 7. Tốc độ học và độ ổn định 25
Câu 8. Tương tác nhiều agent và tính không dừng 25
Câu 9. Hạn chế và độ bền vững 27
Câu 10. Thiết kế agent tốt hơn 29

6 Thảo luận, hạn chế và đề xuất **32**
6.1 Phân biệt quan sát và giải thích 32
6.2 Các nhận xét chung 32
6.3 Hạn chế của chính báo cáo này 33
6.4 Đề xuất cải tiến 33

7 Kết luận **34**

Danh sách bảng

2.1 Lợi nhuận kỳ vọng một bước của agent i theo nhu cầu d và hành động hiện tại b của đối thủ (output của chương trình). 3

3.1 Các thành phần chính của chương trình. 5
3.2 Tham số tham chiếu và cấu hình tái lập. 6

4.1 Kết quả đánh giá baseline (seed 42, 10.000 bước đánh giá). 13
4.2 Thưởng trung bình mỗi bước trong huấn luyện theo cửa sổ 500 episode (seed 42). 14
4.3 Ảnh hưởng của tốc độ học (10 seed). "Tối ưu" là số seed có chính sách cuối đạt 35,00 kỳ vọng; "Đổi chính sách" là số lần chính sách tham lam thay đổi trong 2.500 episode cuối. 16
4.4 Ảnh hưởng của khám phá (10 seed). Thưởng huấn luyện là trung bình 5.000 episode. 18
4.5 Bảng kết quả theo gợi ý của đề bài (đánh giá tham lam, 10 seed, 10.000 bước mỗi seed). 19
4.6 So sánh từng cặp RL–luật trên cùng chuỗi nhu cầu (`multi_seed_summary.csv`). 20

5.1 So sánh agent RL và agent dựa trên luật. 22
5.2 Đối thủ cố định và hai agent cùng học (đánh giá tham lam, 10 seed). 26
5.3 Giá trị kỳ vọng chính xác mỗi bước của các chính sách mốc trong từng kịch bản. 28
5.4 Lợi nhuận đánh giá mỗi bước (TB $\pm$ SD, 10 seed) và số seed đạt tối ưu. Kịch bản doanh thu báo lợi nhuận thực tế, không phải doanh thu. 28
5.5 Regret = tối ưu $-$ giá trị kỳ vọng chính xác (TB 10 seed). Kịch bản doanh thu tính theo đơn vị doanh thu. 31

6.1 Quan sát đo được và mức độ chắc chắn của giải thích. 32

Danh sách hình

2.1 Vòng tương tác của hai agent trong một bước thời gian. 2

4.1 Thí nghiệm B: thưởng mỗi bước theo episode, trung bình cửa sổ $K = 100$, tỷ lệ giảm giá của RL và ε (seed 42). 15
4.2 Giá trị kỳ vọng chính xác của chính sách tham lam trong lúc huấn luyện (seed 42). 15
4.3 Quỹ đạo tám giá trị Q cuối mỗi episode (seed 42). 16
4.4 Thí nghiệm C: đường học (TB $\pm$ SD trên 10 seed, cửa sổ 100 episode). 17
4.5 Thí nghiệm C: hiệu $Q((0,0),0) - Q((0,0),1)$ theo episode, một lần chạy với seed 1042 cho mỗi α. 17
4.6 Thí nghiệm C: thưởng đánh giá theo α. 17
4.7 Thí nghiệm D: đường học theo cấu hình khám phá (TB $\pm$ SD trên 10 seed). . . 18
4.8 Thí nghiệm D: thưởng đánh giá theo cấu hình khám phá. 19
4.9 Thí nghiệm E: RL và luật trên cùng chuỗi nhu cầu theo từng seed. 20

5.1 Câu 8: thưởng và tỷ lệ giảm giá trong huấn luyện khi đối thủ cố định và khi hai agent cùng học (TB 10 seed). 26
5.2 Câu 9–10: lợi nhuận đánh giá khi thị trường thay đổi. 28

Phần 1

Giới thiệu và mục tiêu

Thí nghiệm Agent 02 so sánh hai cách xây dựng agent ra quyết định trong cùng một môi trường: **agent học tăng cường** (RL) dùng Q-learning dạng bảng, và **agent dựa trên luật** (rule-based) dùng chính sách if–else do người viết sẵn [1, 2]. Hai người bán cùng bán một loại sản phẩm; ở mỗi bước thời gian, mỗi người quyết định *giảm giá* hoặc *không giảm giá*. Giảm giá làm tăng xác suất khách mua nhưng làm giảm lợi nhuận trên mỗi sản phẩm, nên agent phải cân bằng hai tác động này.

Mục tiêu của báo cáo:

1. Mô hình hóa bài toán theo các khái niệm agent, môi trường, trạng thái, hành động, phần thưởng và chính sách; giải thích vì sao đây là hệ nhiều agent.
2. Cài đặt môi trường, agent Q-learning và agent dựa trên luật bằng Python, tách riêng giai đoạn huấn luyện và giai đoạn đánh giá.
3. Thực hiện năm thí nghiệm A–E của đề bài: chạy baseline, xem quá trình học, thay đổi tốc độ học, thay đổi mức khám phá và so sánh công bằng trên nhiều seed.
4. Trả lời mười câu hỏi bằng số liệu đo được; cài đặt và đánh giá một thay đổi môi trường (câu 9) và một agent cải tiến (câu 10).

Mã nguồn trên GitHub: `https://github.com/KhoiF/PTHTTM`. Chương trình nằm tại `src/Agent02/agent02_experiment.py`; toàn bộ kết quả (CSV, PNG, bảng Q và nhật ký chạy `run_log.txt`) nằm trong `src/Agent02/agent02_results/`. Mọi con số trong báo cáo được lấy từ các tệp này, không có số liệu ước đoán. Các đoạn code trong báo cáo được trích tự động từ tệp nguồn khi biên dịch LaTeX, số dòng ở lề trái là số dòng trong tệp.

Kết luận chính. Trong môi trường tham chiếu (giảm giá 20%), chính sách tối ưu khi đối thủ là agent dựa trên luật là *không bao giờ giảm giá*, đạt lợi nhuận kỳ vọng 35,00 mỗi bước. Agent dựa trên luật cũng không bao giờ giảm giá trong tình huống này nên đã đạt đúng mức tối ưu. Agent RL học được chính sách tối ưu ở 9/10 seed, đạt trung bình $34{,}72 \pm 0{,}75$; chênh lệch so với agent dựa trên luật không có ý nghĩa thống kê. Khi đổi mức giảm giá xuống 5%, luật cố định không còn tối ưu và RL vượt luật (36,72 so với 34,99). Agent dựa trên mô hình đề xuất ở câu 10 gần đạt tối ưu trong cả sáu kịch bản thử.

Phần 2

Môi trường và hai agent

2.1 Trạng thái, hành động và động lực môi trường

Với agent $i \in \{1, 2\}$, trạng thái cục bộ tại thời điểm t là

$$s_i(t) = (d_t,\ c_i(t)), \qquad d_t \in \{0, 1\},\ c_i(t) = a_{-i}(t-1) \in \{0, 1\}, \tag{2.1}$$

trong đó d_t là nhu cầu thị trường (0: thấp, 1: cao) và $c_i(t)$ là hành động của đối thủ ở bước trước. Mỗi agent có 4 trạng thái và 2 hành động $a_i(t) \in \{0, 1\}$ (0: không giảm giá, 1: giảm giá). Hai agent chọn hành động *đồng thời*, nên không agent nào biết hành động hiện tại của đối thủ.

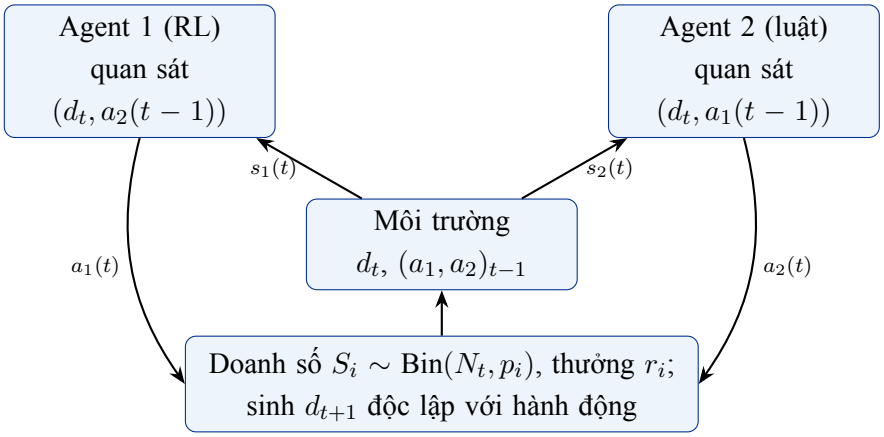
Xác suất một khách tiềm năng mua hàng của agent i là

$$p_i(t) = \mathrm{clip}\big(0{,}30 + 0{,}25 d_t + 0{,}15 a_i(t) - 0{,}10 a_{-i}(t),\ 0{,}05,\ 0{,}95\big). \tag{2.2}$$

Số khách tiềm năng là $N_t = 10$ khi $d_t = 0$ và $N_t = 20$ khi $d_t = 1$. Số đơn bán được $S_i(t) \sim \mathrm{Binomial}(N_t, p_i(t))$. Với giá gốc $P = 10$, giá vốn $C = 5$ và mức giảm $\delta = 0{,}20$, phần thưởng là lợi nhuận

$$r_i(t) = S_i(t)\big[P(1 - \delta a_i(t)) - C\big]. \tag{2.3}$$

Sau khi trả thưởng, môi trường sinh d_{t+1} ngẫu nhiên đều, độc lập với hành động, và hành động hiện tại trở thành "hành động trước" của bước sau. Mỗi episode có 20 bước; đầu episode, nhu cầu được rút ngẫu nhiên và hành động trước của cả hai agent được đặt bằng 0.



Hình 2.1: Vòng tương tác của hai agent trong một bước thời gian.

2.2 Agent A: Q-learning dạng bảng

Agent RL giữ bảng $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ gồm $4 \times 2 = 8$ giá trị. Sau mỗi bước, giá trị của cặp trạng thái–hành động vừa dùng được cập nhật

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \Big], \tag{2.4}$$

với $\alpha = 0{,}10$, $\gamma = 0{,}90$; ở bước cuối episode, mục tiêu chỉ còn r_t. Hành động được chọn theo ε-greedy: ngẫu nhiên với xác suất ε, ngược lại chọn $\arg\max_a Q(s_t, a)$ (hòa thì chọn ngẫu nhiên). ε giảm theo $\varepsilon_e = \max(0{,}02;\ 0{,}20 \cdot 0{,}995^e)$ ở episode e.

2.3 Agent B: chính sách dựa trên luật

$$\pi_{\text{rule}}(d, c) = \begin{cases} 0, & d = 0, \\ 1, & d = 1 \text{ và } c = 1, \\ 0, & d = 1 \text{ và } c = 0. \end{cases} \tag{2.5}$$

Nghĩa là: nhu cầu thấp thì không giảm giá; nhu cầu cao thì giảm giá chỉ khi đối thủ vừa giảm giá ở bước trước. Agent này không dùng phần thưởng để thay đổi hành vi.

2.4 Phân tích lợi nhuận kỳ vọng một bước

Trước khi chạy thí nghiệm, có thể tính lợi nhuận kỳ vọng $\mathbb{E}[r_i] = N_t\, p_i\, (P(1 - \delta a_i) - C)$ cho mọi tổ hợp. Lợi nhuận mỗi sản phẩm là $10 - 5 = 5$ khi không giảm giá và $8 - 5 = 3$ khi giảm giá 20%.

Bảng 2.1: Lợi nhuận kỳ vọng một bước của agent i theo nhu cầu d và hành động hiện tại b của đối thủ (output của chương trình).

d	$b = a_{-i}(t)$	Không giảm giá	Giảm giá	Hành động tốt hơn
0	0	$10 \cdot 0{,}30 \cdot 5 = 15{,}0$	$10 \cdot 0{,}45 \cdot 3 = 13{,}5$	không giảm
0	1	$10 \cdot 0{,}20 \cdot 5 = 10{,}0$	$10 \cdot 0{,}35 \cdot 3 = 10{,}5$	giảm
1	0	$20 \cdot 0{,}55 \cdot 5 = 55{,}0$	$20 \cdot 0{,}70 \cdot 3 = 42{,}0$	không giảm
1	1	$20 \cdot 0{,}45 \cdot 5 = 45{,}0$	$20 \cdot 0{,}60 \cdot 3 = 36{,}0$	không giảm

Agent dựa trên luật không bao giờ giảm giá khi nhu cầu thấp, nên dòng thứ hai (nơi giảm giá có lợi hơn) không xảy ra khi đối thủ là agent luật. Ngoài ra, nếu agent RL giảm giá ở bước t và nhu cầu ở bước $t + 1$ cao, agent luật sẽ giảm giá ở bước $t + 1$, khiến lợi nhuận của RL giảm từ 55 xuống 45. Vì vậy không giảm giá vừa có lợi trước mắt vừa không kích hoạt phản ứng của đối thủ. Khi đối thủ là agent luật, chính sách tối ưu là *luôn không giảm giá*, cho lợi nhuận kỳ

vọng $0{,}5 \cdot 15 + 0{,}5 \cdot 55 = 35{,}0$ mỗi bước. Hàm `best_response` trong chương trình xác nhận điều này bằng cách liệt kê toàn bộ 16 chính sách tất định trên trạng thái (d, c) (Mục 3.4.5).

Một hệ quả quan trọng: hai agent luật đối đầu nhau cũng không bao giờ giảm giá (vì hành động trước ban đầu là 0), nên agent luật đặt vào ghế của RL đạt đúng 35,0. Trong môi trường tham chiếu, RL *tốt nhất chỉ có thể hòa* agent dựa trên luật.

Phần 3

Cài đặt và thiết lập thí nghiệm

3.1 Kiến trúc chương trình

Chương trình phát triển từ mã tham khảo `agent02_experiment.py` của đề bài [2], giữ nguyên mô hình thị trường và các tham số, và thêm các phần cần cho thí nghiệm A–E và câu 8–10.

Bảng 3.1: Các thành phần chính của chương trình.

Thành phần	Vai trò
`Config`	Tham số mô phỏng, seed, số episode, kích thước cửa sổ
`ECommerceEnvironment`	Sinh nhu cầu, doanh số, phần thưởng và trạng thái tiếp theo
`QLearningAgent`	Q-learning với trạng thái (d, c) như đề bài
`ExtendedQLearningAgent`	Q-learning với trạng thái $(d, c, a_i(t-1))$ (so sánh ở câu 10)
`ModelBasedAgent`	Agent đề xuất ở câu 10: học mô hình rồi lập kế hoạch
`RuleBasedAgent`, `TitForTatAgent`	Agent luật của đề bài và đối thủ thay thế cho câu 9–10
`run_training`, `train_rl_against_rule`	Vòng huấn luyện chung cho mọi cặp agent
`evaluate_pair`	Đánh giá tách biệt: đóng băng bảng Q, $\varepsilon = 0$, seed riêng
`exact_evaluation`, `best_response`	Giá trị kỳ vọng chính xác của chính sách và chính sách tối ưu
`run_baseline`, `run_*_experiment`	Thí nghiệm A–E, câu 8, câu 9–10; ghi CSV và hình

3.2 Các thay đổi so với mã tham khảo

Đề bài cho phép sửa mã nếu ghi lại thay đổi và giữ so sánh hợp lệ, tái lập được [1]. Các thay đổi gồm:

M1. Số ngẫu nhiên chung. Môi trường dùng một bộ sinh số cho nhu cầu và một bộ riêng cho doanh số của từng agent, cùng tách từ một seed bằng `SeedSequence`. Khi đánh giá RL–

luật và luật–luật cùng seed, hai lần chạy thấy *đúng cùng một chuỗi nhu cầu*; nếu hai agent chọn cùng hành động thì doanh số cũng giống hệt. Nhờ vậy chênh lệch đo được đến từ chính sách, không phải từ may rủi của chuỗi nhu cầu.

M2. Giao diện agent thống nhất (`observe`, `act`, `learn`). Cùng một vòng huấn luyện và đánh giá dùng được cho RL–luật, RL–RL và các agent mới. Mỗi agent tự tạo quan sát từ trạng thái chung $(d_t, a_1(t-1), a_2(t-1))$.

M3. Đánh giá chính xác. Hàm `exact_evaluation` lan truyền phân phối của trạng thái chung qua 20 bước và tính lợi nhuận kỳ vọng của chính sách tham lam mà không cần lấy mẫu. Hàm `best_response` liệt kê mọi chính sách tất định để tìm mức tối ưu cho từng cách biểu diễn trạng thái.

M4. Thí nghiệm bổ sung: đường học theo cửa sổ, ε cố định, 10 seed độc lập, hai agent cùng học, thay đổi mức giảm giá, hàm thưởng và đối thủ, agent cải tiến.

3.3 Tham số và quy trình đánh giá

Bảng 3.2: Tham số tham chiếu và cấu hình tái lập.

Tham số	Giá trị
Số agent	2
Số episode huấn luyện × số bước	$5.000 \times 20 = 100.000$ bước
Tốc độ học α, hệ số chiết khấu γ	0,10; 0,90
Khám phá ε	0,20 → 0,02, hệ số giảm 0,995 mỗi episode
Giá gốc, giá vốn, mức giảm	10; 5; 20%
Seed baseline (môi trường)	42; seed của agent RL $= 42 + 10.000$
Seed đánh giá baseline	$42 + 500.000$
Seed nhiều lần chạy	11, 22, 33, 44, 55, 66, 77, 88, 99, 110
Seed đánh giá nhiều lần chạy	seed huấn luyện $+800.000$
Đánh giá	500 episode × 20 bước $= 10.000$ bước, $\varepsilon = 0$, bảng Q đóng băng
Môi trường phần mềm	Python 3.12.13, NumPy 2.5.2, Matplotlib 3.11.1
Thời gian chạy toàn bộ	khoảng 40 giây trên MacBook (CPU)

Các chỉ số đo: tổng thưởng, thưởng trung bình mỗi bước $\bar{R}_i = \frac{1}{T}\sum_{t=1}^{T} r_i(t)$, số lần giảm/không giảm giá, tỷ lệ giảm giá ρ_i, bảng Q và chính sách tham lam. Với nhiều seed, báo cáo dùng trung bình và độ lệch chuẩn mẫu $(n-1)$. Chạy lại chương trình hai lần cho nhật ký giống hệt nhau, nghĩa là kết quả tái lập hoàn toàn với cùng môi trường phần mềm.

Chuỗi chính sách như `0100` liệt kê hành động tham lam ở các trạng thái theo thứ tự $(0,0), (0,1), (1,0), (1,1)$ của (d, c); 1 là giảm giá.

3.4 Trích đoạn mã nguồn chính

3.4.1 Mô hình thị trường và môi trường

Hàm xác suất chuyển đổi cài đặt công thức (2.2); `step` nhận hai hành động đồng thời, trả phần thưởng (2.3) rồi sinh nhu cầu mới độc lập.

```
def conversion_probability(demand, own_action, competitor_action):
    """p_i(t) = clip(0.30 + 0.25 d + 0.15 a_i - 0.10 a_-i, 0.05, 0.95)."""
    p = (
        0.30
        + 0.25 * demand
        + 0.15 * own_action
        - 0.10 * competitor_action
    )
    return min(max(p, 0.05), 0.95)


def effective_price(config, action):
    return config.price * (1.0 - config.discount_rate * action)


def profit_per_sale(config, action):
    return effective_price(config, action) - config.unit_cost


def reward_per_sale(config, action):
    if config.reward_mode == "revenue":
        return effective_price(config, action)
    return profit_per_sale(config, action)


def customers_for(config, demand):
    if demand == 0:
        return config.low_demand_customers
    return config.high_demand_customers
```

```
    def __init__(self, config, seed=None):
        self.config = config
        actual_seed = config.seed if seed is None else seed

        # M1: separate streams for demand and for each agent's sales.
        streams = np.random.SeedSequence(actual_seed).spawn(3)
        self.demand_rng = np.random.default_rng(streams[0])
        self.sales_rngs = [
            np.random.default_rng(stream) for stream in streams[1:]
        ]

        self.demand = 0
        self.previous_actions = (0, 0)

    def reset(self):
        self.demand = int(self.demand_rng.integers(0, 2))
        self.previous_actions = (0, 0)

    def step(self, actions):
        if len(actions) != 2:
            raise ValueError("Exactly two actions are required.")

        if any(action not in (0, 1) for action in actions):
            raise ValueError("Actions must be 0 or 1.")

        customers = customers_for(self.config, self.demand)

        rewards = []
        profits = []
        sales_list = []

        # Both actions are selected before this function is called.
```

```
        # Neither agent observes the other's current action beforehand.
        for i in range(2):
            conversion = conversion_probability(
                self.demand, actions[i], actions[1 - i]
            )

            sales = int(self.sales_rngs[i].binomial(customers, conversion))

            rewards.append(sales * reward_per_sale(self.config, actions[i]))
            profits.append(sales * profit_per_sale(self.config, actions[i]))
            sales_list.append(sales)

        # Store current actions for the next observation.
        self.previous_actions = tuple(actions)

        # Demand evolves independently of current actions.
        self.demand = int(self.demand_rng.integers(0, 2))

        info = {
            "sales": sales_list,
            "profits": profits,
            "customers": customers
        }

        return rewards, info
```

3.4.2 Agent Q-learning

`act` cài đặt ε-greedy với phá hòa ngẫu nhiên; `learn` cài đặt công thức (2.4).

```
    def observe(self, demand, previous_actions, i):
        return (demand, previous_actions[1 - i])

    def set_episode(self, episode):
        # Decrease exploration as training progresses.
        self.epsilon = max(
            self.epsilon_min,
            self.epsilon_start * self.epsilon_decay ** episode
        )

    def finalize(self):
        pass

    def act(self, state, explore=True):
        # Exploration.
        if explore and self.rng.random() < self.epsilon:
            return int(self.rng.integers(0, 2))

        # Exploitation with random tie-breaking.
        q0, q1 = self.q[state]

        if math.isclose(q0, q1, rel_tol=1e-9, abs_tol=1e-12):
            return int(self.rng.integers(0, 2))

        return int(q1 > q0)

    def discount_probability(self, state):
        """Probability of action 1 under the greedy evaluation policy."""
        q0, q1 = self.q[state]

        if math.isclose(q0, q1, rel_tol=1e-9, abs_tol=1e-12):
            return 0.5

        return float(q1 > q0)

    def learn(self, state, action, reward, next_state, terminal=False):
        old_value = self.q[state + (action,)]

        if terminal:
            target = reward
        else:
```

```
            target = reward + self.gamma * np.max(self.q[next_state])

        # Q-learning update.
        self.q[state + (action,)] = old_value + self.alpha * (
            target - old_value
        )
```

3.4.3 Agent dựa trên luật

```
class RuleBasedAgent:
    """
    Fixed policy:

    Rule 1: Low demand -> no discount.
    Rule 2: High demand and competitor previously discounted
            -> offer a discount.
    Rule 3: High demand and competitor did not previously discount
            -> no discount.

    This agent does not learn from rewards.
    """

    learns = False

    def observe(self, demand, previous_actions, i):
        return (demand, previous_actions[1 - i])

    def set_episode(self, episode):
        pass

    def finalize(self):
        pass

    def act(self, state, explore=False):
        demand, competitor_previous_action = state

        if demand == 0:
            return 0

        elif demand == 1 and competitor_previous_action == 1:
            return 1

        else:
            return 0

    def discount_probability(self, state):
        return float(self.act(state))
```

3.4.4 Vòng huấn luyện

Hai agent quan sát và chọn hành động trước khi môi trường thay đổi; chỉ agent có `learns = True` cập nhật bảng Q.

```
    for episode in range(config.episodes):

        for agent in agents:
            agent.set_episode(episode)

        env.reset()

        episode_rewards = [0.0, 0.0]
        episode_discounts = [0, 0]

        for step in range(T):

            # Both agents choose actions before the environment changes.
```

```
            states = [
                agent.observe(env.demand, env.previous_actions, i)
                for i, agent in enumerate(agents)
            ]

            actions = [
                agent.act(states[i], explore=True)
                for i, agent in enumerate(agents)
            ]

            rewards, info = env.step(actions)

            terminal = step == T - 1

            for i, agent in enumerate(agents):
                if agent.learns:
                    next_state = agent.observe(
                        env.demand, env.previous_actions, i
                    )
                    agent.learn(
                        states[i], actions[i], rewards[i],
                        next_state, terminal
                    )

                episode_rewards[i] += rewards[i]
                episode_discounts[i] += actions[i]
```

3.4.5 Đánh giá tách biệt và đánh giá chính xác

`evaluate_pair` chạy chính sách đã đóng băng với $\varepsilon = 0$ trên seed riêng. `exact_evaluation` tính kỳ vọng chính xác: ở mỗi bước, với mỗi trạng thái chung và mỗi tổ hợp hành động, cộng thưởng kỳ vọng $N_t p_i \cdot m_i$ (m_i là biên lợi nhuận) có trọng số xác suất, rồi chuyển khối xác suất sang trạng thái kế tiếp.

```
def evaluate_pair(config, agents, episodes=None, seed=900_000):
    """
    Evaluate two frozen policies.

    Important:
        - No Q-table updates occur.
        - RL exploration is disabled.
        - Evaluation uses a separate environment random seed.
    """

    if episodes is None:
        episodes = config.evaluation_episodes

    env = ECommerceEnvironment(config, seed=seed)
    T = config.steps_per_episode

    total_rewards = np.zeros(2)
    total_profits = np.zeros(2)
    action_counts = np.zeros((2, 2), dtype=int)

    episode_results = []

    for episode in range(episodes):

        env.reset()
        rewards_this_episode = [0.0, 0.0]

        for _ in range(T):
            states = [
                agent.observe(env.demand, env.previous_actions, i)
                for i, agent in enumerate(agents)
            ]

            # Greedy policy: no exploration.
```

```
            actions = [
                agent.act(states[i], explore=False)
                for i, agent in enumerate(agents)
            ]

            rewards, info = env.step(actions)

            for i in range(2):
                total_rewards[i] += rewards[i]
                total_profits[i] += info["profits"][i]
                rewards_this_episode[i] += rewards[i]
                action_counts[i, actions[i]] += 1

        episode_results.append({
            "episode": episode + 1,
            "agent1_reward": rewards_this_episode[0],
            "agent2_reward": rewards_this_episode[1],
            "agent1_mean_reward_per_step": rewards_this_episode[0] / T,
            "agent2_mean_reward_per_step": rewards_this_episode[1] / T
        })

    total_steps = episodes * T

    summary = []

    for i in range(2):
        summary.append({
            "seat": AGENT_NAMES[i],
            "total_reward": total_rewards[i],
            "mean_reward_per_step": total_rewards[i] / total_steps,
            "mean_profit_per_step": total_profits[i] / total_steps,
            "no_discount_actions": int(action_counts[i, 0]),
            "discount_actions": int(action_counts[i, 1]),
            "discount_action_rate": action_counts[i, 1] / total_steps
        })

    return summary, episode_results
```

```
def exact_evaluation(config, agents):
    """
    Expected mean reward/profit per step and discount rate of two
    frozen greedy policies under the evaluation protocol:
    T-step episodes, uniform initial demand, previous actions (0, 0).

    The joint state (d, a_1(t-1), a_2(t-1)) is Markov, so its
    distribution is propagated step by step without sampling.
    """

    T = config.steps_per_episode

    distribution = {(0, 0, 0): 0.5, (1, 0, 0): 0.5}

    reward = np.zeros(2)
    profit = np.zeros(2)
    discount = np.zeros(2)

    for _ in range(T):
        next_distribution = defaultdict(float)

        for (demand, p0, p1), mass in distribution.items():
            previous = (p0, p1)
            customers = customers_for(config, demand)

            probs = [
                agent.discount_probability(
                    agent.observe(demand, previous, i)
                )
                for i, agent in enumerate(agents)
            ]

            for actions in product((0, 1), repeat=2):
                weight = mass
                for i in range(2):
                    weight *= probs[i] if actions[i] else 1.0 - probs[i]
```

```
                if weight == 0.0:
                    continue

                for i in range(2):
                    sales = customers * conversion_probability(
                        demand, actions[i], actions[1 - i]
                    )
                    reward[i] += weight * sales * reward_per_sale(
                        config, actions[i]
                    )
                    profit[i] += weight * sales * profit_per_sale(
                        config, actions[i]
                    )
                    discount[i] += weight * actions[i]

                for next_demand in (0, 1):
                    next_distribution[
                        (next_demand,) + actions
                    ] += 0.5 * weight

        distribution = next_distribution

    return {
        "reward": reward / T,
        "profit": profit / T,
        "discount_rate": discount / T
    }


def best_response(config, opponent, agent_class=QLearningAgent):
    """
    Best deterministic stationary policy for seat 0 over the state
    representation of agent_class, found by exhaustive enumeration
    (16 policies for the reference state, 256 for the extended one).
    """

    prototype = agent_class()
    states = prototype.states()

    best_value = -np.inf
    best_table = None

    for bits in product((0, 1), repeat=len(states)):
        table = dict(zip(states, bits))
        policy = TablePolicyAgent(table, prototype.observe)

        value = exact_evaluation(config, [policy, opponent])["reward"][0]

        if value > best_value + 1e-9:
            best_value = value
            best_table = table

    return best_table, best_value
```

Phần 4

Kết quả thí nghiệm

4.1 Thí nghiệm A: baseline

Agent RL được huấn luyện 5.000 episode với seed 42 trước agent luật, sau đó được đánh giá riêng trong 500 episode. Để so sánh công bằng, agent luật cũng được đặt vào đúng ghế của RL và chạy với cùng seed đánh giá.

```
Separate evaluation results (500 episodes, seed 500042):
RL vs Rule    | RL (Q-learning)  | total =  345565.0 | mean reward/step = 34.5565 | discount
↪  =     0 | no discount = 10000
RL vs Rule    | Rule-based       | total =  346720.0 | mean reward/step = 34.6720 | discount
↪  =     0 | no discount = 10000
Rule vs Rule  | Rule-based       | total =  345565.0 | mean reward/step = 34.5565 | discount
↪  =     0 | no discount = 10000
Rule vs Rule  | Rule-based       | total =  346720.0 | mean reward/step = 34.6720 | discount
↪  =     0 | no discount = 10000

Expected reward/step (exact): RL greedy = 35.0000, rule in seat 1 = 35.0000,
best (d, c) policy = 35.0000 {(0, 0): 0, (0, 1): 0, (1, 0): 0, (1, 1): 0}
```

Bảng 4.1: Kết quả đánh giá baseline (seed 42, 10.000 bước đánh giá).

Lần chạy	Agent	Tổng thưởng	TB/bước	Giảm giá	Không giảm
RL vs Luật	RL (ghế 1)	345.565	34,5565	0	10.000
RL vs Luật	Luật (ghế 2)	346.720	34,6720	0	10.000
Luật vs Luật	Luật (ghế 1)	345.565	34,5565	0	10.000
Luật vs Luật	Luật (ghế 2)	346.720	34,6720	0	10.000

Chính sách tham lam học được là *không giảm giá ở cả bốn trạng thái*. Agent luật cũng không giảm giá lần nào, nên hai lần chạy cho kết quả trùng đến từng đơn vị nhờ dùng chung số ngẫu nhiên (M1). Chênh lệch giữa ghế 1 (34,56) và ghế 2 (34,67) chỉ do doanh số ngẫu nhiên khác nhau; giá trị kỳ vọng của cả hai là 35,00.

```
Q-table: Q[d, c, action]; Action 0 = no discount; Action 1 = discount
State=(0, 0)  | Q0=209.034622 | Q1=200.012813 | Greedy action=0
```

```
State=(0, 1) | Q0=227.101684 | Q1=181.167889 | Greedy action=0
State=(1, 0) | Q0=266.318271 | Q1=228.504950 | Greedy action=0
State=(1, 1) | Q0=228.334394 | Q1=219.318896 | Greedy action=0
```

Mọi trạng thái có $Q_0 > Q_1$ nên chính sách tham lam chọn không giảm giá. Độ chênh nhỏ nhất (khoảng 9) nằm ở $(0, 0)$ và $(1, 1)$. Ở $(0, 0)$, điều này khớp với Bảng 2.1: khi nhu cầu thấp, giảm giá chỉ thiệt 1,5 mỗi bước. Trạng thái $(1, 1)$ ít được thăm (đối thủ chỉ giảm giá sau khi RL giảm giá) nên ước lượng của nó dựa trên ít dữ liệu. Giá trị Q vào khoảng 200–270 vì nó ước lượng tổng thưởng chiết khấu của nhiều bước còn lại, gần $35/(1 - 0{,}9) = 350$ ở đầu episode và chỉ còn khoảng 35 ở bước cuối.

4.2 Thí nghiệm B: quá trình học

Bảng 4.2 tính $\bar{R}_{i,k} = \sum_{t \in W_k} r_i(t)/(20K)$ với cửa sổ $K = 500$ episode; Hình 4.1 dùng $K = 100$.

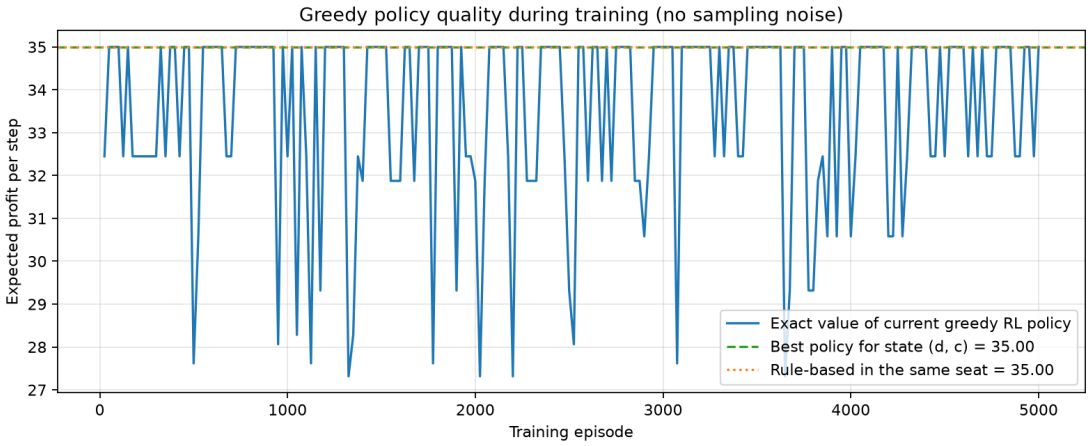
Bảng 4.2: Thưởng trung bình mỗi bước trong huấn luyện theo cửa sổ 500 episode (seed 42).

Episode	RL	Luật	Tỷ lệ giảm giá của RL	ε đầu cửa sổ
1–500	33,474	32,862	0,193	0,200
501–1000	33,412	33,172	0,139	0,020
1001–1500	33,382	32,620	0,195	0,020
1501–2000	33,541	32,922	0,201	0,020
2001–2500	33,015	32,439	0,201	0,020
2501–3000	33,965	33,284	0,170	0,020
3001–3500	33,841	33,472	0,146	0,020
3501–4000	32,492	31,774	0,232	0,020
4001–4500	33,321	32,849	0,179	0,020
4501–5000	33,713	33,177	0,131	0,020

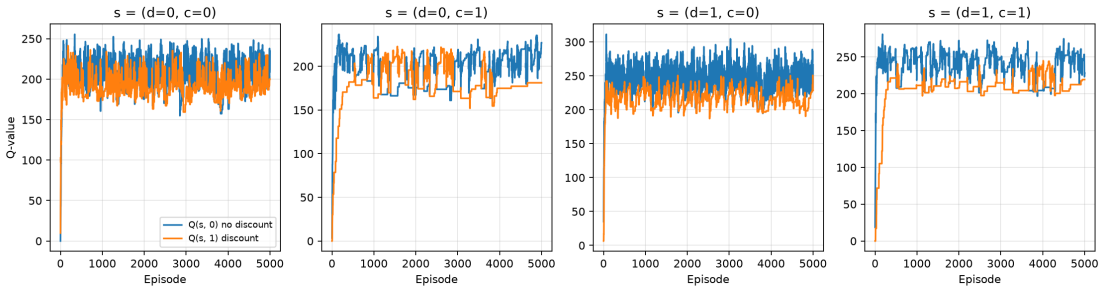


Hình 4.1: Thí nghiệm B: thưởng mỗi bước theo episode, trung bình cửa sổ $K = 100$, tỷ lệ giảm giá của RL và ε (seed 42).

Quan sát. Thưởng huấn luyện của RL dao động quanh 33–34 suốt 5.000 episode và không tăng rõ rệt sau vài trăm episode đầu. Tỷ lệ giảm giá trong huấn luyện vẫn ở mức 0,13–0,23 dù ε đã chạm sàn 0,02 từ khoảng episode 460. Phần khám phá ngẫu nhiên chỉ giải thích được khoảng 0,01 trong tỷ lệ này, nên phần lớn các lần giảm giá đến từ chính sách tham lam tạm thời chọn giảm giá. Hình 4.2 xác nhận điều đó: giá trị kỳ vọng chính xác của chính sách tham lam, đo mỗi 25 episode, liên tục nhảy giữa 35,0 (tối ưu) và các mức 27–33.



Hình 4.2: Giá trị kỳ vọng chính xác của chính sách tham lam trong lúc huấn luyện (seed 42).



Hình 4.3: Quỹ đạo tám giá trị Q cuối mỗi episode (seed 42).

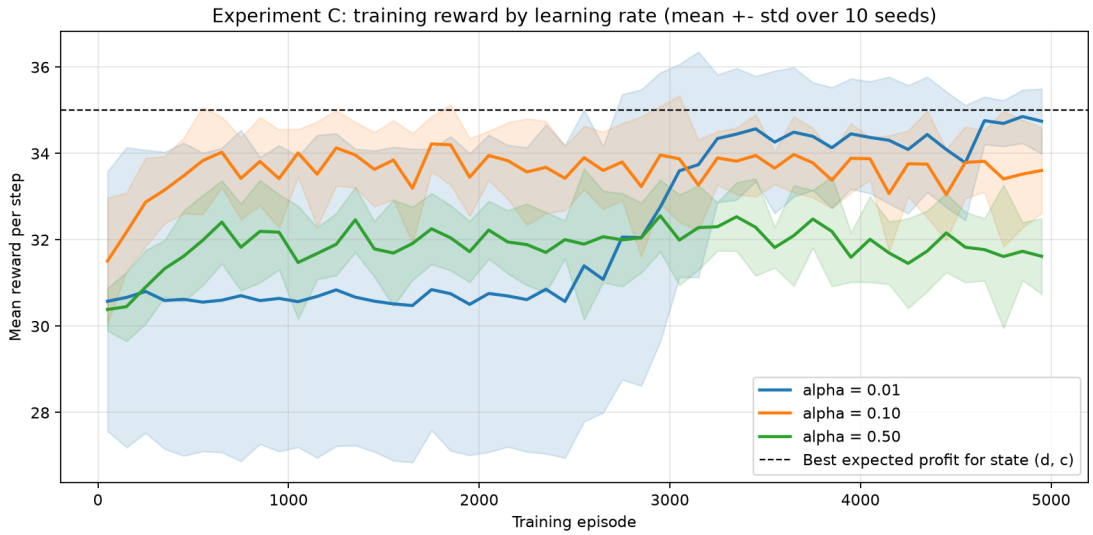
Giải thích. Hình 4.3 cho thấy các giá trị Q không hội tụ mà dao động với biên độ vài chục đơn vị, lớn hơn khoảng cách giữa Q_0 và Q_1 ở trạng thái $(0, 0)$. Có ba nguyên nhân hợp lý: (i) doanh số là biến nhị thức nên thưởng mỗi bước có độ lệch chuẩn lớn (khoảng 11 khi nhu cầu cao); (ii) $\alpha = 0{,}1$ cố định nên mỗi cập nhật chỉ làm trung bình trên khoảng 10 mẫu gần nhất; (iii) trạng thái không chứa vị trí bước trong episode, nên cùng một trạng thái có mục tiêu khoảng 300 ở đầu episode nhưng chỉ $r_t \approx 35$ ở bước cuối. Vì vậy, thưởng huấn luyện tăng hay giảm *không* đủ để kết luận về hội tụ: trong lần chạy này, thưởng huấn luyện gần như không đổi, nhưng chính sách cuối cùng vẫn tối ưu. Chính sách cuối cùng tối ưu một phần nhờ may mắn: nếu dừng ở episode khác, chính sách có thể kém hơn.

4.3 Thí nghiệm C: tốc độ học

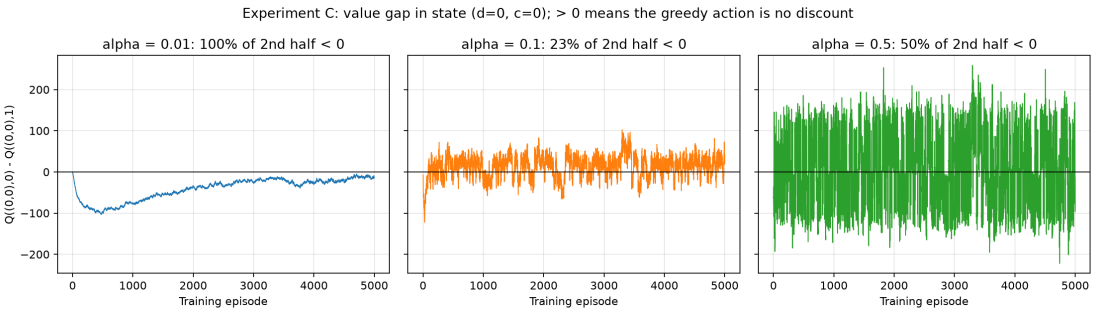
Ba cấu hình $\alpha \in \{0{,}01; 0{,}10; 0{,}50\}$ được chạy với 10 seed; các tham số khác giữ nguyên.

Bảng 4.3: Ảnh hưởng của tốc độ học (10 seed). "Tối ưu" là số seed có chính sách cuối đạt 35,00 kỳ vọng; "Đổi chính sách" là số lần chính sách tham lam thay đổi trong 2.500 episode cuối.

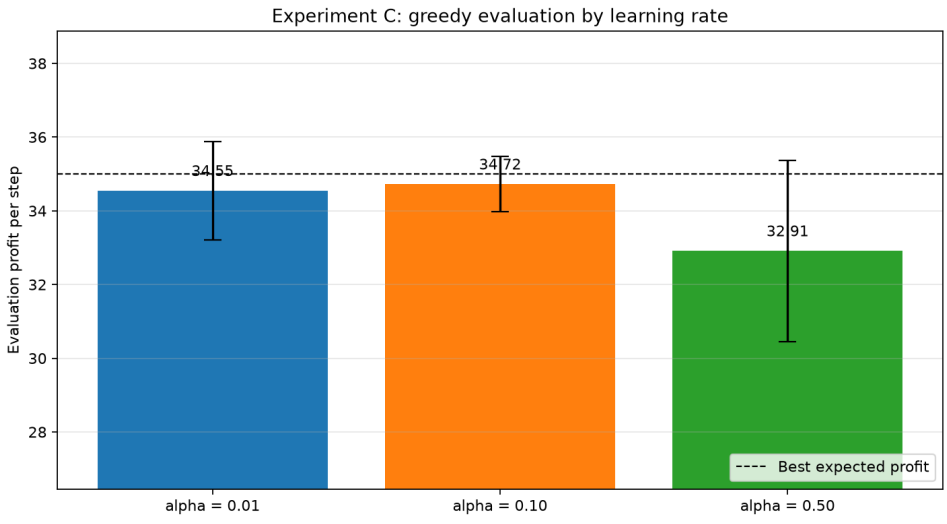
α	Đánh giá (TB $\pm$ SD)	Giảm giá	Huấn luyện	1.000 ep. cuối	Đổi chính sách	Tối ưu
0,01	$34{,}546 \pm 1{,}328$	0,049	32,245	34,414	17,1	9/10
0,10	$34{,}724 \pm 0{,}747$	0,040	33,600	33,566	256,3	9/10
0,50	$32{,}912 \pm 2{,}457$	0,259	31,876	31,763	827,7	5/10



Hình 4.4: Thí nghiệm C: đường học (TB $\pm$ SD trên 10 seed, cửa sổ 100 episode).



Hình 4.5: Thí nghiệm C: hiệu $Q((0,0),0) - Q((0,0),1)$ theo episode, một lần chạy với seed 1042 cho mỗi α.



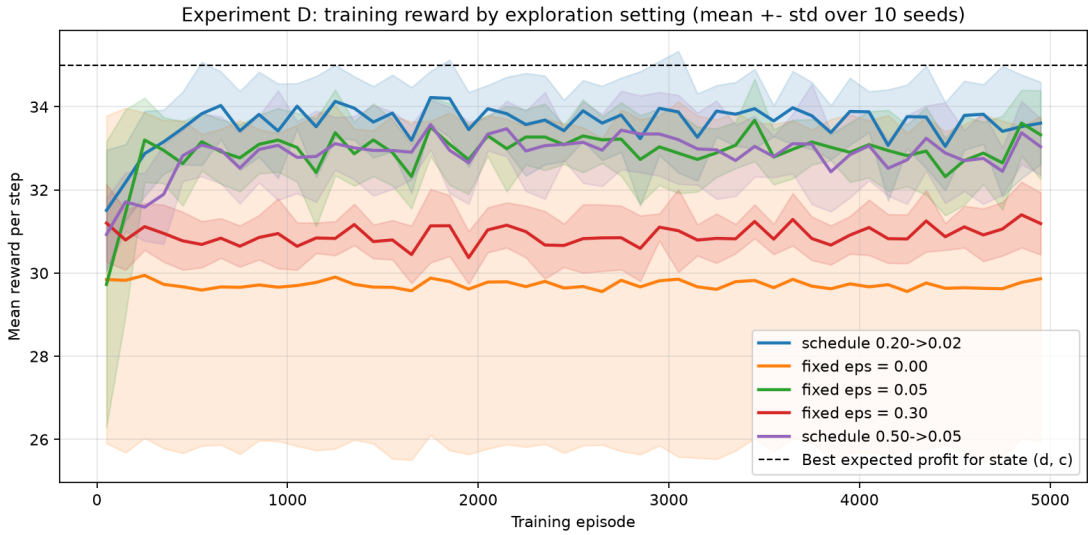
Hình 4.6: Thí nghiệm C: thưởng đánh giá theo α.

4.4 Thí nghiệm D: mức khám phá

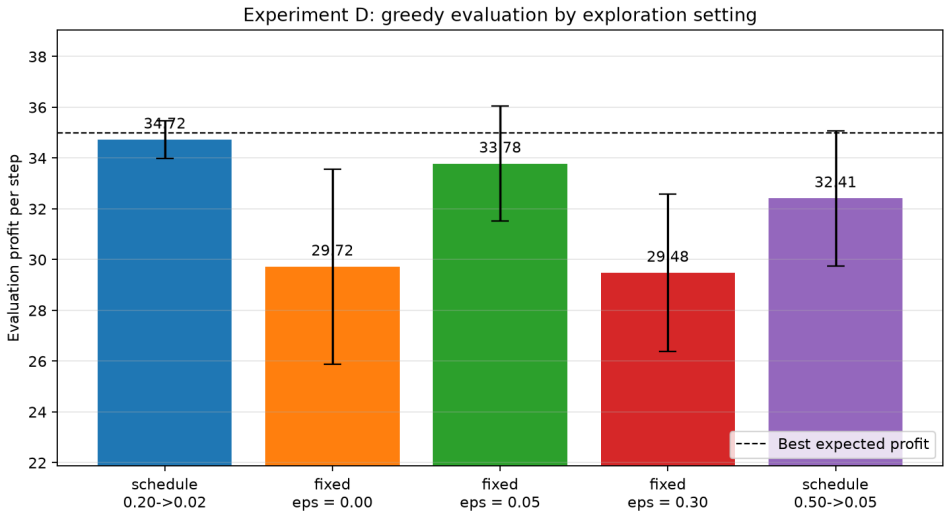
Năm cấu hình được so sánh: lịch tham chiếu $0{,}20 \rightarrow 0{,}02$; ε cố định 0; 0,05; 0,30; và lịch cao $0{,}50 \rightarrow 0{,}05$ của tài liệu code [2]. Cấu hình $\varepsilon = 0$ được thêm để xem điều gì xảy ra khi agent ngừng khám phá ngay từ đầu.

Bảng 4.4: Ảnh hưởng của khám phá (10 seed). Thưởng huấn luyện là trung bình 5.000 episode.

Cấu hình	Đánh giá (TB $\pm$ SD)	Giảm giá (ĐG)	Huấn luyện	Giảm giá (HL)	Đổi CS	Tối ưu
Lịch $0{,}20 \rightarrow 0{,}02$	$34{,}724 \pm 0{,}747$	0,040	33,600	0,167	256,3	9/10
Cố định 0,00	$29{,}719 \pm 3{,}852$	0,415	29,720	0,414	0,0	3/10
Cố định 0,05	$33{,}775 \pm 2{,}267$	0,130	32,907	0,231	502,3	7/10
Cố định 0,30	$29{,}476 \pm 3{,}104$	0,461	30,910	0,400	1.217,0	2/10
Lịch $0{,}50 \rightarrow 0{,}05$	$32{,}411 \pm 2{,}656$	0,300	32,867	0,236	505,2	4/10



Hình 4.7: Thí nghiệm D: đường học theo cấu hình khám phá (TB $\pm$ SD trên 10 seed).



Hình 4.8: Thí nghiệm D: thưởng đánh giá theo cấu hình khám phá.

4.5 Thí nghiệm E: so sánh công bằng trên nhiều seed

RL (lịch tham chiếu) và agent luật được đánh giá trong các lần chạy tách biệt, cùng ghế 1, cùng đối thủ là agent luật và cùng seed đánh giá (seed huấn luyện $+800.000$).

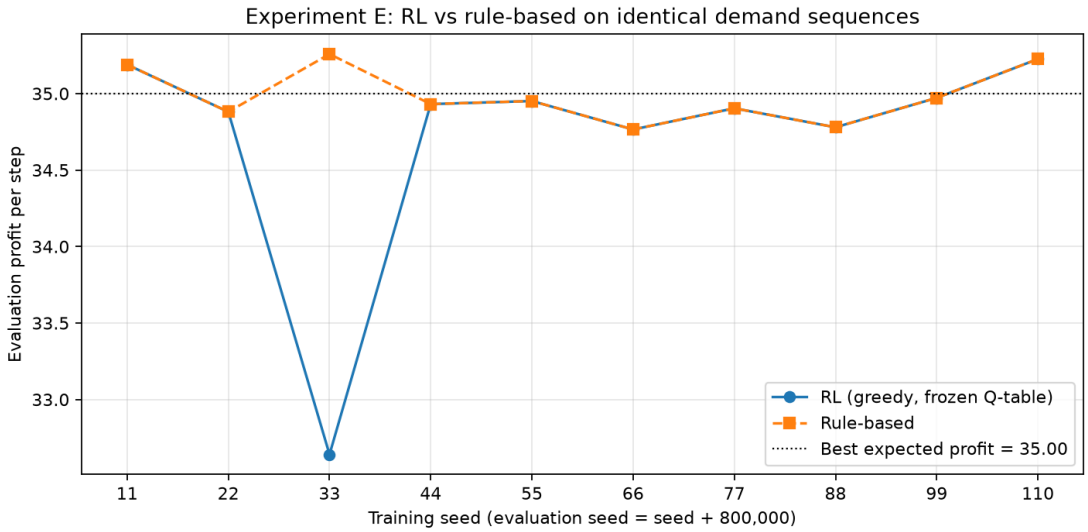
Bảng 4.5: Bảng kết quả theo gợi ý của đề bài (đánh giá tham lam, 10 seed, 10.000 bước mỗi seed).

Agent / cấu hình	Thưởng TB	Độ lệch chuẩn	Tỷ lệ giảm giá
RL, baseline ($\alpha = 0{,}10$, lịch $0{,}20 \to 0{,}02$)	34,724	0,747	0,040
RL, tốc độ học thấp ($\alpha = 0{,}01$)	34,546	1,328	0,049
RL, tốc độ học cao ($\alpha = 0{,}50$)	32,912	2,457	0,259
RL, khám phá khác ($\varepsilon = 0{,}05$ cố định)	33,775	2,267	0,130
RL, khám phá khác ($\varepsilon = 0{,}30$ cố định)	29,476	3,104	0,461
Agent dựa trên luật	34,985	0,178	0,000

Bảng 4.6: So sánh từng cặp RL–luật trên cùng chuỗi nhu cầu (`multi_seed_summary.csv`).

Seed	RL	Luật	RL − Luật	Chính sách RL
11	35,189	35,189	0,000	0100
22	34,881	34,881	0,000	0100
33	32,641	35,258	−2,617	1000
44	34,931	34,931	0,000	0000
55	34,951	34,951	0,000	0000
66	34,765	34,765	0,000	0100
77	34,904	34,904	0,000	0100
88	34,780	34,780	0,000	0000
99	34,972	34,972	0,000	0000
110	35,226	35,226	0,000	0100

Ở 9/10 seed, RL và luật cho kết quả *giống hệt*. Các chính sách `0100` chọn giảm giá ở trạng thái $(0, 1)$, nhưng trạng thái này không bao giờ xuất hiện khi RL không giảm giá: agent luật chỉ giảm giá sau khi RL giảm giá. Vì vậy hành vi thực tế của chúng vẫn là không bao giờ giảm giá. Seed 33 học sai ở trạng thái $(0, 0)$: giảm giá khi nhu cầu thấp, làm lợi nhuận giảm còn 32,64. Hiệu cặp RL−luật có trung bình $-0{,}262$, độ lệch chuẩn $0{,}828$, khoảng tin cậy 95% (phân phối t, 9 bậc tự do) là $[-0{,}854;\ 0{,}330]$, có chứa 0.



Hình 4.9: Thí nghiệm E: RL và luật trên cùng chuỗi nhu cầu theo từng seed.

Phần 5

Trả lời mười câu hỏi

Câu 1. Mô hình hóa agent và môi trường

- **Agent:** hai người bán. Agent 1 là agent Q-learning, agent 2 là agent dựa trên luật (ở câu 8, cả hai đều là agent Q-learning).
- **Môi trường:** thị trường thương mại điện tử gồm nhu cầu d_t, số khách N_t, xác suất mua (2.2) và cơ chế bán theo phân phối nhị thức.
- **Không gian trạng thái** của mỗi agent: $s_i = (d, c) \in \{0, 1\}^2$, gồm 4 trạng thái. Trạng thái đầy đủ của môi trường là $(d_t, a_1(t-1), a_2(t-1))$, gồm 8 trạng thái; mỗi agent chỉ thấy một phần.
- **Không gian hành động:** $\mathcal{A} = \{0, 1\}$ (không giảm/giảm giá), chọn đồng thời.
- **Hàm thưởng:** lợi nhuận (2.3), ngẫu nhiên vì S_i là biến nhị thức.
- **Chuyển trạng thái:** d_{t+1} đều trên $\{0, 1\}$, độc lập với hành động; $c_i(t+1) = a_{-i}(t)$. Mỗi episode có 20 bước và đặt lại hành động trước về 0.

Đây là hệ *nhiều agent* vì phần thưởng và trạng thái của mỗi agent phụ thuộc vào hành động của agent kia: p_i giảm 0,10 khi đối thủ giảm giá, và $c_i(t+1)$ chính là hành động của đối thủ. Nếu coi đối thủ là một phần của môi trường thì môi trường không còn cố định với agent RL khi đối thủ cũng học (câu 8), và ngay cả với đối thủ luật, môi trường phản ứng với hành động của RL (giảm giá kéo theo đối thủ giảm giá). Mô hình một agent sẽ bỏ qua cả tương tác chiến lược này lẫn câu hỏi về lợi ích của từng bên.

Câu 2. Quyết định dựa trên học và dựa trên luật

Bảng 5.1: So sánh agent RL và agent dựa trên luật.

Tiêu chí	Agent RL (Q-learning)	Agent dựa trên luật
Ra quyết định	$\arg\max_a Q(s,a)$, có khám phá khi huấn luyện	Tra bảng if–else cố định
Dùng kinh nghiệm	Cập nhật Q sau mỗi bước từ thưởng quan sát được	Không dùng thưởng; tri thức nằm trong luật do người viết
Khả năng thích ứng	Tự đổi chính sách khi thị trường đổi; ở mức giảm 5% đạt 36,72 so với 34,99 của luật	Phải viết lại luật; ở mức giảm 5% vẫn giữ nguyên hành vi
Độ ổn định	Phụ thuộc seed, α, ε; SD 0,75 (baseline)	Hoàn toàn nhất quán; SD 0,18 chỉ do nhiễu doanh số
Tính toán	100.000 bước huấn luyện ($\approx$ 0,5 s), bảng 8 giá trị	Không cần huấn luyện, $O(1)$ mỗi quyết định
Giải thích được	Cần đọc bảng Q; khó hơn khi trạng thái lớn	Đọc trực tiếp từ luật

Khi nên dùng agent luật: khi có chính sách được kiểm chứng hoặc ràng buộc pháp lý rõ ràng, ví dụ quy định không được bán dưới giá vốn hay chương trình khuyến mãi theo lịch cố định trong dịp lễ. Khi đó tính nhất quán và khả năng giải thích quan trọng hơn vài phần trăm lợi nhuận. Trong thí nghiệm này, luật tham chiếu tình cờ là chính sách tối ưu nên RL không thể làm tốt hơn. **Khi nên dùng agent RL:** khi điều kiện thị trường thay đổi và không ai biết trước luật tốt nhất, ví dụ chi phí, độ nhạy giá hoặc mức giảm giá thay đổi theo mùa. Kịch bản giảm 5% ở câu 9 cho thấy RL tự phát hiện rằng luôn giảm giá có lợi hơn, trong khi luật cũ mất khoảng 2,35 lợi nhuận mỗi bước.

Câu 3. Cập nhật Q-learning

Công thức (2.4) dịch $Q(s,a)$ một phần α về phía *mục tiêu TD* $r + \gamma \max_{a'} Q(s',a')$. Mục tiêu này là ước lượng một bước của tổng thưởng chiết khấu: thưởng nhận ngay cộng giá trị tốt nhất có thể đạt từ trạng thái kế tiếp. Hiệu $\delta = \text{mục tiêu} - Q(s,a)$ là *sai số TD*: dương thì giá trị cũ đang bị đánh giá thấp. α quyết định mức tin vào mẫu mới; γ quyết định trọng số của tương lai. Toán tử max làm Q-learning học *off-policy*: nó ước lượng giá trị của chính sách tham lam dù hành động đang thực hiện có thể là khám phá.

Với $Q(s,a) = 4$, $\alpha = 0{,}1$, $r = 6$, $\gamma = 0{,}9$, $\max_{a'} Q(s',a') = 5$:

$$\begin{aligned}
\text{Mục tiêu TD} &= r + \gamma \max_{a'} Q(s',a') = 6 + 0{,}9 \times 5 = 6 + 4{,}5 = 10{,}5,\\
\text{Sai số TD } \delta &= 10{,}5 - 4 = 6{,}5,\\
\alpha\,\delta &= 0{,}1 \times 6{,}5 = 0{,}65,\\
Q_{\text{mới}}(s,a) &= 4 + 0{,}65 = 4{,}65.
\end{aligned}$$

Chương trình in cùng kết quả: `Q3: target = 6.0 + 0.9 * 5.0 = 10.50; TD error = 6.50; new Q = 4.0 + 0.1 * 6.50 = 4.65`. **Diễn giải:** kinh nghiệm mới cho thấy hành động a tốt hơn mức agent đang tin (10,5 so với 4), nhưng với $\alpha = 0{,}1$ agent chỉ dịch 10% quãng đường, từ 4 lên 4,65. Nếu cùng mục tiêu lặp lại, Q tiến dần về 10,5 theo cấp số nhân với hệ số $(1-\alpha)$; một mẫu nhiễu đơn lẻ không làm thay đổi đánh giá quá mạnh.

Câu 4. Khám phá và khai thác

ε-greedy chọn hành động ngẫu nhiên với xác suất ε để tiếp tục thu thập thông tin về những hành động có giá trị chưa chắc chắn; phần còn lại khai thác hành động có Q cao nhất. Bảng 4.4 và Hình 4.7 cho thấy:

- **Ngừng khám phá quá sớm** ($\varepsilon = 0$): đánh giá $29{,}72 \pm 3{,}85$, chỉ 3/10 seed tối ưu, chính sách không đổi lần nào trong nửa sau. Lý do: mọi lợi nhuận đều dương và Q khởi tạo bằng 0, nên hành động được thử đầu tiên ở mỗi trạng thái (chọn ngẫu nhiên do hòa) có ngay $Q > 0$ và được chọn mãi mãi. Agent bị khóa vào lựa chọn ngẫu nhiên ban đầu; tỷ lệ giảm giá 0,415 gần với kết quả tung đồng xu.
- **Khám phá quá nhiều** ($\varepsilon = 0{,}30$ cố định): thưởng huấn luyện chỉ 30,91 vì khoảng 15% bước là giảm giá ngẫu nhiên, mỗi lần làm mất lợi nhuận và kích hoạt phản ứng giảm giá của đối thủ. Đáng chú ý, chính sách tham lam cuối cùng cũng kém (29,48, 2/10 tối ưu). Một giải thích phù hợp với dữ liệu: trạng thái (d,c) không chứa hành động trước của chính RL, nên giá trị Q mà Q-learning hội tụ tới phụ thuộc vào tần suất RL tự giảm giá trong lúc huấn luyện. Khám phá nhiều làm thay đổi phân phối đó. Thí nghiệm hiện tại chưa đủ để chứng minh cơ chế này; cần một thí nghiệm riêng với trạng thái đầy đủ.
- **Lịch giảm dần** $0{,}20 \to 0{,}02$ cho kết quả tốt nhất (34,72, 9/10), vì khám phá nhiều lúc bảng Q còn trống, rồi khai thác khi đã có ước lượng. $\varepsilon = 0{,}05$ cố định đứng thứ hai (33,78, 7/10);

lịch cao $0{,}50 \to 0{,}05$ kém hơn (32,41, 4/10).

Thưởng huấn luyện và thưởng đánh giá đo hai thứ khác nhau: với $\varepsilon = 0{,}30$, thưởng huấn luyện (30,91) cao hơn thưởng đánh giá (29,48) dù đánh giá không có hành động ngẫu nhiên. Vì vậy chỉ thưởng đánh giá mới phản ánh chất lượng chính sách.

Câu 5. Phần thưởng và mục tiêu kinh doanh

Giảm giá làm tăng xác suất mua thêm 0,15 (công thức (2.2)), nhưng giá bán giảm từ 10 xuống $10 \times (1 - 0{,}20) = 8$ trong khi giá vốn vẫn là 5:

$$\text{Không giảm giá: } 10 - 5 = 5 \text{ trên mỗi sản phẩm,}$$
$$\text{Giảm giá 20\%: } 10 \times 0{,}8 - 5 = 8 - 5 = 3 \text{ trên mỗi sản phẩm.}$$

Biên lợi nhuận giảm 40%, nên giảm giá chỉ có lợi khi số đơn tăng hơn $5/3 \approx 1{,}67$ lần. Khi nhu cầu thấp, tỷ lệ tăng là $0{,}45/0{,}30 = 1{,}5$; khi nhu cầu cao là $0{,}70/0{,}55 \approx 1{,}27$. Cả hai đều dưới 1,67, nên giảm giá làm giảm lợi nhuận kỳ vọng (Bảng 2.1).

Thay đổi hàm thưởng sẽ thay đổi hành vi học được. Chương trình chạy thêm kịch bản phần thưởng là *doanh thu* $S_i \cdot P(1 - \delta a_i)$ thay cho lợi nhuận (Bảng 5.4). Với doanh thu, giảm giá khi nhu cầu thấp cho $10 \times 0{,}45 \times 8 = 36$ so với $10 \times 0{,}30 \times 10 = 30$, tức có lợi trước mắt. Agent RL tham chiếu nâng tỷ lệ giảm giá từ 0,04 lên 0,50, và lợi nhuận thực tế giảm từ 34,72 xuống $29{,}07 \pm 3{,}20$. Agent tối ưu doanh thu nhưng làm doanh nghiệp mất lợi nhuận. Một quan sát phụ: lợi nhuận mỗi đơn với mức giảm 10% là $(5; 4)$, đúng bằng một nửa doanh thu mỗi đơn với mức giảm 20% là $(10; 8)$. Vì Q-learning không đổi khi nhân thưởng với hằng số dương, hai kịch bản này cho cùng chính sách ở mọi seed (cùng tỷ lệ giảm giá 0,500 và cùng 338,5 lần đổi chính sách). Các thay đổi khác cũng làm đổi hành vi: phạt mỗi lần giảm giá sẽ đẩy agent về phía không giảm giá; thêm thưởng cho thị phần hay khách hàng trung thành có thể làm giảm giá trở nên có lợi. Hàm thưởng phải phản ánh đúng mục tiêu kinh doanh, vì agent sẽ tối ưu chính xác thứ được đo.

Câu 6. Đánh giá hiệu năng

Thưởng trung bình mỗi bước $\bar{R}_i = \frac{1}{T}\sum_t r_i(t)$ trong đánh giá tách biệt:

- Baseline seed 42, $T = 10.000$: RL $345.565/10.000 = 34{,}557$; agent luật ở cùng ghế cũng $34{,}557$; agent luật ở ghế đối thủ $34{,}672$ (Bảng 4.1).
- 10 seed: RL $34{,}724 \pm 0{,}747$; luật $34{,}985 \pm 0{,}178$ (Bảng 4.5); giá trị kỳ vọng chính xác lần lượt $34{,}745 \pm 0{,}807$ và $35{,}000$.

Một lần chạy không đủ để kết luận một phương pháp tốt hơn vì: (i) *nhiễu môi trường*: ngay cả khi chính sách giống hệt, ghế 1 và ghế 2 lệch nhau 0,12 chỉ do doanh số ngẫu nhiên, và giữa các seed đánh giá, cùng một chính sách "không giảm giá" cho kết quả từ 34,77 đến 35,26; (ii) *nhiễu huấn luyện*: chính sách cuối phụ thuộc seed; 9 seed hòa với luật nhưng seed 33 thua 2,62.

Nếu chỉ chạy seed 33, ta sẽ kết luận RL kém hẳn; nếu chỉ chạy seed 11 thì kết luận ngược lại; (iii) *phụ thuộc cấu hình*: kết quả đổi theo α, ε và tham số thị trường (câu 9). Với 10 seed và so sánh từng cặp, khoảng tin cậy 95% của hiệu RL−luật là $[-0{,}854; 0{,}330]$, tức không có bằng chứng một bên tốt hơn. Đây cũng chỉ là kết luận trong mô hình mô phỏng này, không phải cho thị trường thật.

Câu 7. Tốc độ học và độ ổn định

Theo Bảng 4.3 và các Hình 4.4, 4.5:

- $\alpha = 0{,}01$ **(chậm, ổn định):** mỗi mẫu chỉ đóng góp 1%, nên ước lượng ít nhiễu (chỉ 17,1 lần đổi chính sách trong nửa sau, so với 256,3 ở $\alpha = 0{,}1$). Đổi lại, agent học rất chậm: đường học trung bình đứng quanh 30,6 trong khoảng 2.500 episode đầu, sau đó mới tăng lên khoảng 34,8. Vì Q khởi tạo bằng 0 và mọi thưởng đều dương, hành động được thử nhiều hơn lúc đầu sẽ có Q cao hơn trong thời gian dài. Hình 4.5 cho thấy ở seed 1042, hiệu $Q_0 - Q_1$ tại $(0, 0)$ vẫn âm suốt 5.000 episode (dù đang tăng dần về 0), tức agent chưa kịp sửa lựa chọn ban đầu. Đánh giá $34{,}55 \pm 1{,}33$; 9/10 seed tối ưu.
- $\alpha = 0{,}10$ **(tham chiếu):** học nhanh trong vài trăm episode, kết quả tốt nhất ($34{,}72 \pm 0{,}75$; 9/10), nhưng hiệu $Q_0 - Q_1$ vẫn đổi dấu 23% thời gian nửa sau ở seed 1042.
- $\alpha = 0{,}50$ **(nhanh, dao động):** mỗi cập nhật thay một nửa giá trị cũ bằng một mẫu nhiễu, nên Q gần như chỉ phản ánh vài mẫu gần nhất. Hiệu $Q_0 - Q_1$ dao động trong khoảng ± 200 và âm 50% thời gian; chính sách đổi 827,7 lần trong nửa sau. Thưởng đánh giá thấp nhất ($32{,}91 \pm 2{,}46$, chỉ 5/10 tối ưu).

Tóm lại, α lớn làm agent nhạy với quan sát mới nhưng ước lượng nhiễu và chính sách không ổn định; α nhỏ làm ước lượng mượt nhưng cần nhiều dữ liệu hơn. Với thưởng nhiều nhiễu như ở đây, α giảm dần theo số lần thăm (ví dụ $1/N(s, a)$) hoặc ước lượng mô hình bằng trung bình mẫu (câu 10) là lựa chọn tốt hơn α cố định.

Câu 8. Tương tác nhiều agent và tính không dừng

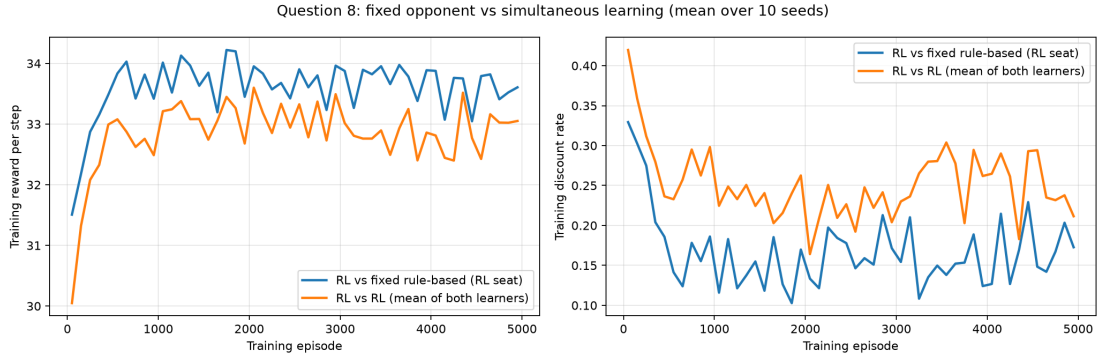
Hành động của đối thủ ảnh hưởng đến agent theo hai đường: (i) *phần thưởng* – khi đối thủ giảm giá, p_i giảm 0,10, ví dụ từ 55 xuống 45 khi nhu cầu cao; (ii) *quan sát* – $c_i(t+1) = a_{-i}(t)$ quyết định agent ở trạng thái nào tiếp theo. Khi đối thủ là agent luật, quan hệ này cố định, nên với agent RL môi trường là dừng (dù không quan sát đầy đủ). Khi *cả hai cùng học*, chính sách đối thủ thay đổi theo thời gian; cùng một cặp (s, a) có phân phối thưởng và trạng thái kế tiếp khác nhau ở các giai đoạn khác nhau. Giả định Markov dừng mà Q-learning cần để hội tụ bị vi phạm: mỗi agent đuổi theo một mục tiêu đang dịch chuyển, và các lần khám phá của agent này trở thành nhiễu trong dữ liệu của agent kia.

Chương trình kiểm tra điều này bằng cách cho hai agent Q-learning độc lập cùng học (10 seed, cùng tham số baseline), rồi so sánh với RL học trước đối thủ cố định:

Bảng 5.2: Đối thủ cố định và hai agent cùng học (đánh giá tham lam, 10 seed).

Thiết lập	Thưởng ĐG (TB ± SD)	Tỷ lệ giảm giá	Đổi CS (nửa sau)
RL vs luật cố định (ghế RL)	34,724 ± 0,747	0,040	256,3
RL vs RL (20 agent)	33,266 ± 2,469	0,187	284,4

```
seed  11 | policies 0100 / 0000 | eval = 35.19 / 35.26 | changes = 271 / 271
seed  44 | policies 1010 / 1101 | eval = 28.73 / 29.90 | changes = 272 / 271
seed  66 | policies 0000 / 1000 | eval = 32.27 / 34.20 | changes = 288 / 297
seed  77 | policies 1100 / 0100 | eval = 33.42 / 32.64 | changes = 247 / 262
seed  99 | policies 0011 / 0100 | eval = 27.31 / 29.62 | changes = 294 / 291
seed 110 | policies 0100 / 1100 | eval = 32.89 / 33.76 | changes = 275 / 296
```



Hình 5.1: Câu 8: thưởng và tỷ lệ giảm giá trong huấn luyện khi đối thủ cố định và khi hai agent cùng học (TB 10 seed).

Khi cùng học, 5/10 seed kết thúc ở cặp chính sách kém (seed 44, 66, 77, 99, 110), so với 1/10 khi đối thủ cố định. Lợi nhuận đánh giá giảm 1,46 mỗi bước, độ lệch chuẩn tăng hơn ba lần, và tỷ lệ giảm giá trong huấn luyện luôn cao hơn (0,20–0,30 so với 0,10–0,20). Ở seed 44 và 99 cả hai agent cùng giảm giá ở nhiều trạng thái, một dạng "chiến tranh giá" nhỏ làm cả hai mất lợi nhuận. Số lần đổi chính sách chỉ tăng nhẹ (284 so với 256); phần lớn dao động đã có sẵn do nhiễu (Thí nghiệm B), nên thí nghiệm này chưa tách riêng được phần do tính không dừng.

Đề xuất thí nghiệm để nghiên cứu sâu hơn: *học luân phiên* – đóng băng agent 2 trong M episode trong khi agent 1 học, rồi đổi vai. Mỗi pha khi đó là bài toán dừng. So sánh với học đồng thời (cùng tổng số bước, cùng seed) về số seed đạt cặp chính sách tốt và tần suất đổi chính sách sẽ cho biết phần suy giảm nào do tính không dừng. Một biến thể khác là đưa hành động trước của chính agent vào trạng thái, hoặc cho agent học trên hành động chung (joint-action learner).

Câu 9. Hạn chế và độ bền vững

Hạn chế của mô phỏng tham chiếu:

1. **Trạng thái không Markov với agent RL.** Agent luật phản ứng với hành động trước của RL, nhưng trạng thái (d, c) của RL không chứa hành động trước của chính nó. Hậu quả của việc giảm giá (đối thủ giảm giá ở bước sau) bị gán cho một trạng thái không liên quan. Trạng thái cũng không chứa số bước còn lại trong episode, trong khi bước cuối dùng mục tiêu khác.
2. **Mô hình chuyển đổi tuyến tính và minh họa:** hệ số 0,15 và 0,10 không được ước lượng từ dữ liệu; nhu cầu chỉ có hai mức và độc lập với hành động (giảm giá không kéo khách quay lại sau đó, không có hiệu ứng mùa vụ).
3. **Chỉ hai hành động và một mức giảm giá;** không có tồn kho, chi phí vận hành, khách hàng trung thành hay khác biệt sản phẩm.
4. **Đối thủ cố định và đối xứng:** RL học trước một luật duy nhất; chính sách học được có thể kém khi đối thủ đổi cách chơi.
5. **Kết luận nhạy với tham số:** chính sách tối ưu đổi hẳn khi mức giảm giá đổi từ 20% xuống 5% (Bảng 5.3).

Thay đổi được cài đặt. Hàm `scenario_list` thay đổi *mức giảm giá* (5%, 10%, 30%), *hàm thưởng* (doanh thu) và *hành vi đối thủ* (tit-for-tat: lặp lại hành động trước của RL bất kể nhu cầu). Mỗi kịch bản được huấn luyện và đánh giá trên 10 seed như Thí nghiệm E; mức tối ưu được tính chính xác.

```
def scenario_list(config):
    return [
        ("discount 20% (reference)", config, RuleBasedAgent),
        ("discount 5%", replace(config, discount_rate=0.05), RuleBasedAgent),
        ("discount 10%", replace(config, discount_rate=0.10), RuleBasedAgent),
        ("discount 30%", replace(config, discount_rate=0.30), RuleBasedAgent),
        ("revenue reward, 20%", replace(config, reward_mode="revenue"),
         RuleBasedAgent),
        ("discount 5%, tit-for-tat", replace(config, discount_rate=0.05),
         TitForTatAgent),
    ]

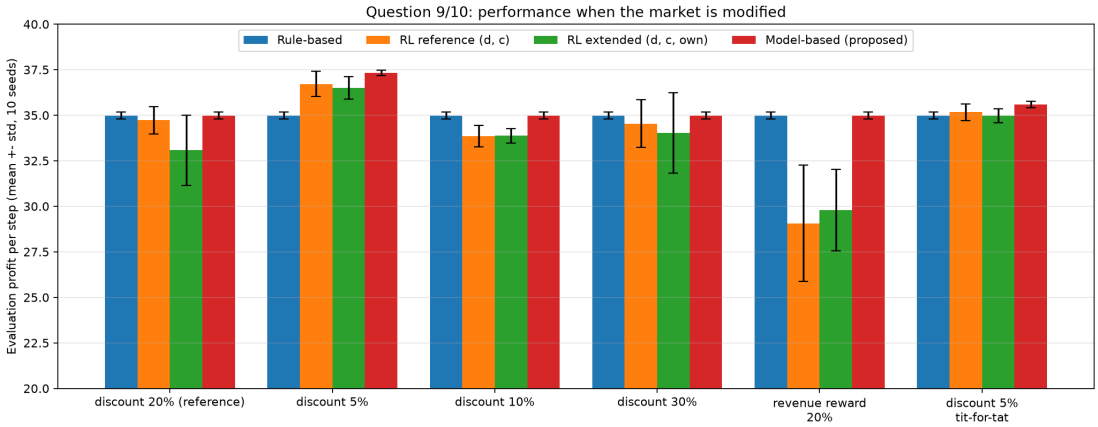

LEARNING_AGENTS = [
    ("RL reference (d, c)", QLearningAgent),
    ("RL extended (d, c, own)", ExtendedQLearningAgent),
    ("Model-based (proposed)", ModelBasedAgent),
]
```

Bảng 5.3: Giá trị kỳ vọng chính xác mỗi bước của các chính sách mốc trong từng kịch bản.

Kịch bản	Không giảm	Luôn giảm	Luật	Tối ưu (d, c)	Chính sách tối ưu
Giảm 20% (tham chiếu)	35,000	24,900	35,000	35,000	0000
Giảm 5%	35,000	37,350	35,000	37,350	1111
Giảm 10%	35,000	33,200	35,000	35,000	0000
Giảm 30%	35,000	16,600	35,000	35,000	0000
Thưởng doanh thu, 20%	70,000	66,400	70,000	70,000	0000
Giảm 5%, đối thủ tit-for-tat	35,000	35,212	35,000	35,675	0011

Bảng 5.4: Lợi nhuận đánh giá mỗi bước (TB $\pm$ SD, 10 seed) và số seed đạt tối ưu. Kịch bản doanh thu báo lợi nhuận thực tế, không phải doanh thu.

Kịch bản	Luật	RL (d, c)	RL (d, c, own)	Dựa trên mô hình
Giảm 20%	$34{,}99 \pm 0{,}18$	$34{,}72 \pm 0{,}75$ (9)	$33{,}07 \pm 1{,}91$ (4)	$34{,}99 \pm 0{,}18$ (10)
Giảm 5%	$34{,}99 \pm 0{,}18$	$36{,}72 \pm 0{,}70$ (2)	$36{,}51 \pm 0{,}62$ (1)	$37{,}33 \pm 0{,}14$ (9)
Giảm 10%	$34{,}99 \pm 0{,}18$	$33{,}85 \pm 0{,}59$ (1)	$33{,}87 \pm 0{,}39$ (1)	$34{,}99 \pm 0{,}18$ (10)
Giảm 30%	$34{,}99 \pm 0{,}18$	$34{,}54 \pm 1{,}32$ (9)	$34{,}03 \pm 2{,}20$ (8)	$34{,}99 \pm 0{,}18$ (10)
Doanh thu, 20%	$34{,}99 \pm 0{,}18$	$29{,}07 \pm 3{,}20$ (1)	$29{,}79 \pm 2{,}24$ (1)	$34{,}99 \pm 0{,}18$ (10)
5%, tit-for-tat	$34{,}99 \pm 0{,}18$	$35{,}16 \pm 0{,}46$ (1)	$34{,}98 \pm 0{,}38$ (0)	$35{,}59 \pm 0{,}18$ (5)



Hình 5.2: Câu 9–10: lợi nhuận đánh giá khi thị trường thay đổi.

Ảnh hưởng của thay đổi.

- **Giảm 5%:** lợi nhuận mỗi đơn khi giảm giá là 4,5. Khi nhu cầu thấp, giảm giá cho $10 \times 0{,}45 \times 4{,}5 = 20{,}25 > 15$; khi nhu cầu cao, $20 \times 0{,}70 \times 4{,}5 = 63 > 55$. Luôn giảm giá trở thành tối ưu (37,35). Agent luật không đổi nên mất 2,35 mỗi bước (6,3%); RL tham chiếu tự chuyển sang giảm giá 75% thời gian và đạt 36,72. Đây là kịch bản cho thấy rõ lợi thế thích ứng của RL.

- **Giảm 10%:** giảm giá có lợi trước mắt (khi nhu cầu thấp 18 so với 15; khi nhu cầu cao 56 so với 55) nhưng kích hoạt phản ứng giảm giá của đối thủ ở bước sau. Tối ưu vẫn là không giảm giá. RL tham chiếu chỉ thấy lợi ích trước mắt, vì hậu quả xảy ra ở trạng thái mà nó không phân biệt được (hạn chế 1). Kết quả là RL giảm giá 50% thời gian và đạt 33,85 – *thua* agent luật.
- **Giảm 30%:** giảm giá thiệt hại rõ (biên lợi nhuận chỉ còn 2), nên RL học đúng 9/10 seed, gần giống kịch bản tham chiếu.
- **Đối thủ tit-for-tat, 5%:** tối ưu là chỉ giảm giá khi nhu cầu cao (35,675). RL tham chiếu đạt 35,16, nhỉnh hơn luật nhưng chỉ 1/10 seed đạt tối ưu, vì khoảng cách giá trị giữa các chính sách rất nhỏ so với nhiễu.

Câu 10. Thiết kế agent tốt hơn

Phân tích ở câu 7–9 chỉ ra hai điểm yếu của agent RL tham chiếu: *trạng thái thiếu thông tin* và *cập nhật TD với α cố định quá nhiễu*. Agent đề xuất (`ModelBasedAgent`) xử lý cả hai.

- **Trạng thái:** $s_i = (d_t,\ a_{-i}(t-1),\ a_i(t-1))$, 8 trạng thái. Thêm hành động trước của chính agent làm trạng thái trở thành Markov khi đối thủ phản ứng theo hành động trước (agent luật, tit-for-tat).
- **Hành động:** $\{0, 1\}$ như đề bài; có thể mở rộng thành nhiều mức giảm giá mà không đổi thuật toán.
- **Hàm thưởng:** lợi nhuận (2.3), đúng mục tiêu kinh doanh (câu 5 cho thấy thưởng doanh thu dẫn tới hành vi sai).
- **Phương pháp học và ra quyết định:** học *mô hình* thay vì học trực tiếp Q. Agent đếm $N(s, a)$, tổng thưởng và tần suất trạng thái kế tiếp để ước lượng $\hat{R}(s, a)$ và $\hat{P}(s' \mid s, a)$. $\hat{P}$ chứa cả quá trình nhu cầu lẫn *mô hình phản ứng của đối thủ*. Cứ 50 episode và khi kết thúc huấn luyện, agent giải $Q(s, a) = \hat{R}(s, a) + \gamma \sum_{s'} \hat{P}(s' \mid s, a) \max_{a'} Q(s', a')$ bằng lặp giá trị với $\gamma = 0{,}98$, gần mục tiêu thưởng trung bình hơn $\gamma = 0{,}9$. Ước lượng bằng trung bình mẫu có phương sai giảm dần theo $1/N$, thay vì giữ mức nhiễu cố định như $\alpha = 0{,}1$. Khám phá dùng cùng lịch ε với baseline để so sánh công bằng.
- **Quy trình đánh giá:** cùng giao thức tách biệt như Thí nghiệm E (10 seed, 500 episode, $\varepsilon = 0$, số ngẫu nhiên chung), cộng giá trị kỳ vọng chính xác và mức tối ưu cho từng kịch bản. Để đo độ bền vững, dùng *regret* = tối ưu − giá trị kỳ vọng, và lấy *regret lớn nhất* trên tập kịch bản đa dạng (mức giảm giá, hàm thưởng, đối thủ). Agent bền vững hơn là agent có regret lớn nhất nhỏ hơn và độ lệch chuẩn giữa các seed nhỏ hơn.

```
342 class ModelBasedAgent(ExtendedQLearningAgent):
343     """
344     Question 10: proposed agent.
345 
346     State: (d, c, own previous action), as in ExtendedQLearningAgent.
347     Learning: instead of a single noisy TD update, the agent counts
```

```
        N(s, a), the reward sum and next-state frequencies, giving
        R_hat(s, a) and P_hat(s' | s, a). P_hat contains the demand
        process and a model of the competitor's reaction.
    Planning: value iteration on the learned model every 50 episodes
        and once after training. plan_gamma = 0.98 is closer to the
        average-reward objective used in evaluation.
    Exploration: same epsilon-greedy schedule as the reference agent.
    """

    plan_gamma = 0.98
    plan_every = 50

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        n_states = int(np.prod(self.state_shape))
        self.visits = np.zeros((n_states, 2))
        self.reward_sum = np.zeros((n_states, 2))
        self.transitions = np.zeros((n_states, 2, n_states))

    def index(self, state):
        return int(np.ravel_multi_index(state, self.state_shape))

    def learn(self, state, action, reward, next_state, terminal=False):
        i = self.index(state)
        self.visits[i, action] += 1
        self.reward_sum[i, action] += reward
        self.transitions[i, action, self.index(next_state)] += 1

    def set_episode(self, episode):
        super().set_episode(episode)

        if episode % self.plan_every == 0:
            self.plan()

    def finalize(self):
        self.plan()

    def plan(self):
        visits = np.maximum(self.visits, 1)
        reward = self.reward_sum / visits
        transition = self.transitions / visits[..., None]

        value = np.zeros(reward.shape[0])

        for _ in range(1000):
            q = reward + self.plan_gamma * transition @ value
            new_value = q.max(axis=1)

            if np.max(np.abs(new_value - value)) < 1e-8:
                break

            value = new_value

        self.q = q.reshape(self.state_shape + (2,))

```

Bảng 5.5: Regret = tối ưu − giá trị kỳ vọng chính xác (TB 10 seed). Kịch bản doanh thu tính theo đơn vị doanh thu.

Kịch bản	Luật	RL (d, c)	RL (d, c, own)	Dựa trên mô hình
Giảm 20%	0,000	0,255	1,887	0,000
Giảm 5%	2,350	0,630	0,868	0,020
Giảm 10%	0,000	1,149	1,121	0,000
Giảm 30%	0,000	0,439	0,952	0,000
Doanh thu, 20%	0,000	2,298	2,242	0,000
5%, tit-for-tat	0,675	0,539	0,717	0,085
Regret lớn nhất	2,350	2,298	2,242	**0,085**

Kết quả kiểm tra. Agent dựa trên mô hình đạt tối ưu ở 10/10 seed trong bốn kịch bản, 9/10 ở kịch bản 5% (trung bình 37,33 so với tối ưu 37,35) và 5/10 với đối thủ tit-for-tat (35,59 so với 35,675). Regret lớn nhất chỉ 0,085, so với 2,35 của luật (thua khi giảm 5%) và khoảng 2,3 của RL tham chiếu (thua khi khoảng cách giá trị nhỏ). Chính sách của nó gần như không đổi trong nửa sau huấn luyện (0–2,3 lần, so với hàng trăm lần của Q-learning).

Một kết quả đối chứng quan trọng: *chỉ thêm trạng thái mà vẫn dùng Q-learning* (`ExtendedQLearningAgent`) không cải thiện, thậm chí kém hơn ở kịch bản tham chiếu (33,07, 4/10). Bảng Q có 16 giá trị nên mỗi ô nhận ít dữ liệu hơn, trong khi nhiễu cập nhật không giảm. Như vậy, cải thiện của agent đề xuất đến từ việc *kết hợp* trạng thái đầy đủ với ước lượng ít nhiễu và lập kế hoạch, không đến từ riêng phần trạng thái.

Giới hạn của đề xuất: agent cần không gian trạng thái nhỏ, rời rạc để đếm; nó giả định đối thủ dừng trong lúc học (với đối thủ cũng đang học, mô hình cần quên dữ liệu cũ, ví dụ dùng cửa sổ trượt); và tập kịch bản kiểm tra ở đây vẫn nhỏ. Một kiểm tra mạnh hơn sẽ thêm đối thủ ngẫu nhiên, đối thủ đổi chiến lược giữa chừng và nhu cầu có tính mùa vụ, rồi so sánh regret lớn nhất của cả ba agent trên tập đó.

Phần 6

Thảo luận, hạn chế và đề xuất

6.1 Phân biệt quan sát và giải thích

Bảng 6.1: Quan sát đo được và mức độ chắc chắn của giải thích.

Quan sát	Giải thích	Mức bằng chứng
RL hòa luật ở 9/10 seed (tham chiếu)	Luật trùng chính sách tối ưu; RL tốt nhất chỉ hòa	Chứng minh bằng liệt kê chính xác
Thưởng huấn luyện không tăng sau $\sim$500 episode	Chính sách tham lam dao động giữa tối ưu và kém	Đo trực tiếp (Hình 4.2)
Dao động lớn của Q	Nhiễu nhị thức, α cố định, trạng thái thiếu số bước còn lại	Hợp lý, chưa tách riêng từng nguyên nhân
$\varepsilon = 0$ khóa vào lựa chọn ban đầu	Thưởng luôn dương, Q khởi tạo 0	Suy ra trực tiếp từ công thức cập nhật
$\varepsilon = 0{,}30$ cho chính sách tham lam kém	Điểm hội tụ phụ thuộc phân phối hành vi khi trạng thái không Markov	Giả thuyết, cần thí nghiệm riêng
RL thua luật khi giảm 10%	RL không thấy hậu quả trễ của giảm giá	Phù hợp với tính toán; agent dựa trên mô hình sửa được
Hai agent cùng học kém hơn	Không dừng và vòng phản hồi giảm giá	Quan sát; chưa tách khỏi nhiễu

6.2 Các nhận xét chung

- **Agent luật** cho quyết định nhất quán (độ lệch chuẩn giữa các seed chỉ 0,18, hoàn toàn do nhiễu doanh số), nhưng hiệu quả phụ thuộc vào việc luật có khớp với môi trường hay không: tối ưu ở mức giảm 10–30%, mất 6,3% lợi nhuận ở mức 5%.
- **Agent RL** cải thiện được qua phản hồi và thích ứng khi thị trường đổi, nhưng việc học không đảm bảo chính sách tối ưu: kết quả phụ thuộc seed, α, ε và cách biểu diễn trạng thái.

- **Nhiễu ngẫu nhiên** làm thưởng dao động ngay cả khi chính sách không đổi (34,77–35,26 giữa các seed đánh giá của cùng chính sách). Vì vậy giá trị kỳ vọng chính xác và số ngẫu nhiên chung rất hữu ích để so sánh.
- **Hai agent cùng học** ảnh hưởng đến kết quả của nhau (câu 8).
- **Huấn luyện và đánh giá đo hai thứ khác nhau:** thưởng huấn luyện gồm chi phí khám phá và chính sách đang thay đổi; thưởng đánh giá đo chính sách cuối với bảng Q đóng băng.

6.3 Hạn chế của chính báo cáo này

- 10 seed đủ để thấy xu hướng nhưng khoảng tin cậy vẫn rộng; các so sánh như $\alpha = 0{,}01$ với $0{,}10$ không tách biệt được về mặt thống kê.
- Agent đề xuất được thiết kế *sau khi* đã thấy điểm yếu của agent tham chiếu trên chính các kịch bản này, nên có nguy cơ thiết kế khớp quá mức với tập kiểm tra. Cần thêm kịch bản mới chưa dùng khi thiết kế để kiểm tra lại.
- Mọi kết luận chỉ đúng trong mô hình mô phỏng đơn giản hóa; mô hình chuyển đổi là minh họa, không được kiểm chứng với dữ liệu thật [2].

6.4 Đề xuất cải tiến

1. Đưa hành động trước của chính agent (và nếu cần, số bước còn lại) vào trạng thái; dùng α giảm theo số lần thăm hoặc học dựa trên mô hình.
2. Mở rộng môi trường: nhu cầu có quán tính (chuỗi Markov), nhiều mức giảm giá, tồn kho, khách hàng trung thành để giảm giá có lợi ích dài hạn.
3. Đánh giá theo regret trên một tập kịch bản và đối thủ đa dạng, báo cáo trung bình, độ lệch chuẩn và trường hợp xấu nhất.
4. Với nhiều agent cùng học: học luân phiên, mô hình hóa đối thủ có quên dữ liệu cũ, hoặc các thuật toán học đa agent như joint-action learner.

Phần 7

Kết luận

Báo cáo đã cài đặt môi trường thương mại điện tử hai người bán, agent Q-learning và agent dựa trên luật, tách riêng giai đoạn huấn luyện và đánh giá, và thực hiện đủ năm thí nghiệm A–E trên 10 seed độc lập cùng ba phần mở rộng cho câu 8–10.

Trong môi trường tham chiếu, kết quả đo được không ủng hộ nhận định RL tốt hơn hay kém hơn luật: luật trùng với chính sách tối ưu (35,00), RL đạt tối ưu ở 9/10 seed và chênh lệch không có ý nghĩa thống kê. Kết quả đổi khi môi trường đổi: ở mức giảm giá 5%, RL thích ứng và vượt luật; ở mức 10%, RL thua luật vì trạng thái không thể hiện hậu quả trễ của việc giảm giá. Tốc độ học và lịch khám phá ảnh hưởng mạnh đến độ ổn định: $\alpha = 0{,}5$ và $\varepsilon = 0{,}30$ cố định cho chính sách kém hơn nhiều; $\varepsilon = 0$ khóa agent vào lựa chọn ngẫu nhiên ban đầu. Khi hai agent cùng học, số seed kết thúc ở chính sách kém tăng từ 1/10 lên 5/10.

Agent đề xuất ở câu 10 – trạng thái có hành động trước của chính agent, học mô hình thị trường và đối thủ bằng đếm, rồi lập kế hoạch – có regret lớn nhất 0,085 trên sáu kịch bản, so với khoảng 2,3 của cả agent luật lẫn agent RL tham chiếu. Bài học chính của thí nghiệm là: chọn agent nào không thể dựa vào một lần chạy với phần thưởng cao nhất; cần một quy trình đánh giá tách biệt, có mốc tối ưu, lặp nhiều seed và thử trên nhiều điều kiện môi trường.

Tài liệu tham khảo

[1] Intelligent System Development, *Laboratory Experiment 02: Reinforcement-Learning-Based Agents versus Rule-Based Agents*, năm học 2026–2027, tệp `TAILIEU/ASSIGNMENT-AGENT_02/assignment_Agent02_due_29.10.pdf`.

[2] Intelligent System Development, *AGENT02: Python Implementation – Reinforcement-Learning-Based Agents and Rule-Based Agents*, tệp `TAILIEU/ASSIGNMENT-AGENT_02/agent02_complementary_code.pdf` và mã tham khảo `agent02_experiment.py`.

[3] R. S. Sutton và A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[4] Phan Văn Khôi, *agent02_experiment.py* và kết quả thí nghiệm, https://github.com/KhoiF/PTHTTM, thư mục `src/Agent02/`.